University of Asia Pacific

Department of Computer Science and Engineering

CSE 450: VLSI Lab

LAB REPORT

Experiment Number: 01

Experiment Title: Design and Simulation of a 1-bit Full Adder Using VHDL

Submitted by:

Name: Nazmul Mridha

Student ID: 22201161

Section: C2

Submitted to:

Md. Rasheduzzaman
Lecturer
Department of Computer Science and Engineering

Date of Submission: 03/10/2026

1. Experiment Name

Design and Simulation of a 1-bit Full Adder using VHDL in Xilinx ISE

2. Objective

The goals of this experiment are:

- To design a 1-bit full adder in VHDL using dataflow (concurrent signal assignment) modelling.
- To verify the design with a test bench that applies all eight combinations of A, B and Cin.
- To generate the RTL schematic with XST synthesis and observe the behavior in ISim behavioral simulation.
- To compare the simulated outputs with the full-adder truth table.

3. Apparatus / Hardware & Software Requirements

- Xilinx ISE Design Suite (XST synthesis and ISim simulator, ISim P.20131013)
- Oracle VirtualBox virtual machine

4. Circuit Diagram / Schematic

A full adder adds three one-bit inputs: the operands A and B and the carry-in Cin. It produces a one-bit Sum and a carry-out Cout. The Boolean equations implemented in the VHDL code are:

$$\mathbf{Sum = A \oplus B \oplus Cin}$$
$$\mathbf{Cout = (A \cdot B) + (Cin \cdot (A \oplus B))}$$

The carry-out uses the shared term A ⊕ B, which is also needed for the Sum. This lets the design reuse one XOR gate and needs only two AND gates and one OR gate for the carry, instead of three AND gates and a three-input OR.

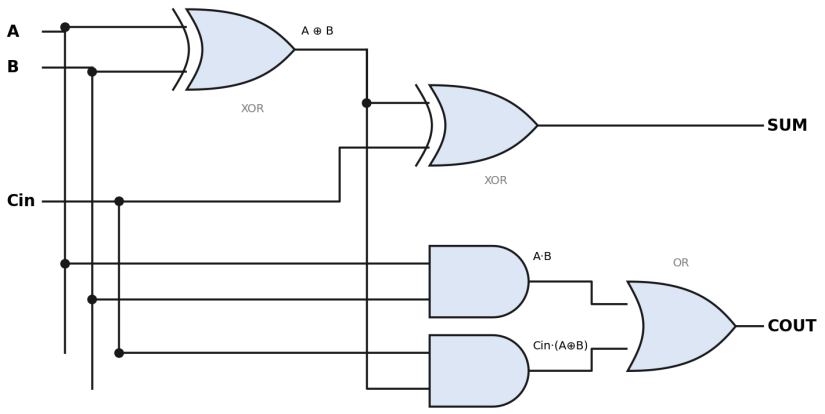


Fig 1: Gate-level diagram of the 1-bit full adder

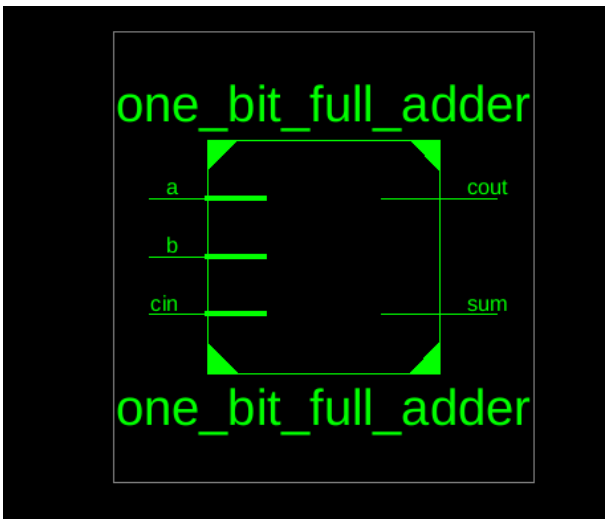


Fig 2: RTL schematic of the 1-bit full adder, top-level block

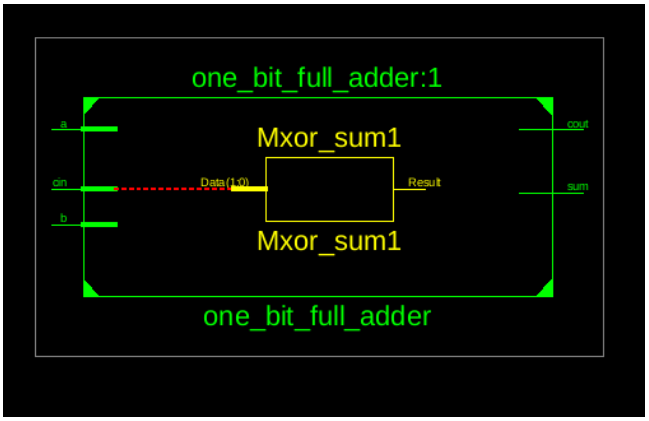


Fig 3: RTL schematic, expanded view showing the Mxor_sum1 block

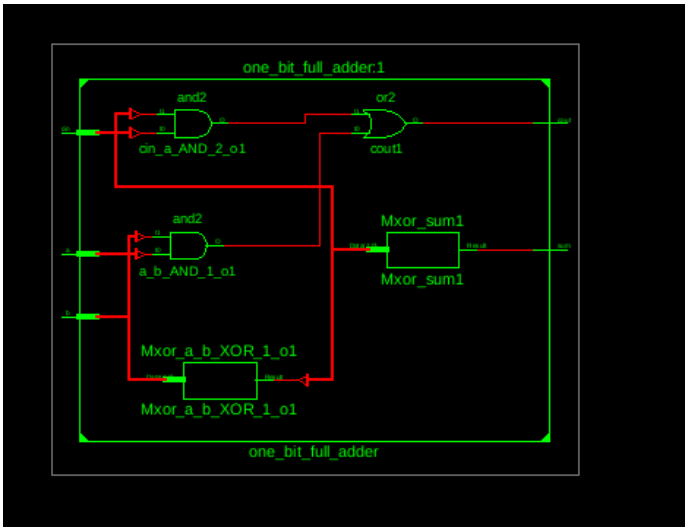


Fig 4: RTL schematic of the 1-bit full adder, internal logic

5. VHDL Source Code

5.1 Design File (one_bit_full_adder.vhd)

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity one_bit_full_adder is
    Port ( a    : in  STD_LOGIC;
           b    : in  STD_LOGIC;
           cin  : in  STD_LOGIC;
           sum  : out STD_LOGIC;
           cout : out STD_LOGIC);
end one_bit_full_adder;

architecture Behavioral of one_bit_full_adder is
begin
    sum  <= a xor b xor cin;
    cout <= (a and b) or (cin and (a xor b));
end Behavioral;
```

5.2 Test Bench (one_bit_full_adder_tb.vhd)

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY one_bit_full_adder_tb IS
END one_bit_full_adder_tb;

ARCHITECTURE behavior OF one_bit_full_adder_tb IS

    -- Component Declaration for the Unit Under Test (UUT)
    COMPONENT one_bit_full_adder
    PORT(
         a    : IN  std_logic;
         b    : IN  std_logic;
         cin  : IN  std_logic;
         sum  : OUT std_logic;
         cout : OUT std_logic
        );
    END COMPONENT;

    --Inputs
    signal a   : std_logic := '0';
    signal b   : std_logic := '0';
    signal cin : std_logic := '0';

    --Outputs
    signal sum  : std_logic;
    signal cout : std_logic;

BEGIN

    -- Instantiate the Unit Under Test (UUT)
    uut: one_bit_full_adder PORT MAP (
          a    => a,
          b    => b,
          cin  => cin,
          sum  => sum,
          cout => cout
        );

    -- Stimulus process
    stim_proc: process
    begin
        a <= '0'; b <= '0'; cin <= '0'; wait for 100 ns;
        a <= '0'; b <= '0'; cin <= '1'; wait for 100 ns;
        a <= '0'; b <= '1'; cin <= '0'; wait for 100 ns;
        a <= '0'; b <= '1'; cin <= '1'; wait for 100 ns;
        a <= '1'; b <= '0'; cin <= '0'; wait for 100 ns;
        a <= '1'; b <= '0'; cin <= '1'; wait for 100 ns;
        a <= '1'; b <= '1'; cin <= '0'; wait for 100 ns;
        a <= '1'; b <= '1'; cin <= '1'; wait for 100 ns;
```

```
        wait;
    end process;

END;
```

6. Output Analysis / Observations

Behavioral simulation was run in ISim with 100 ns per step, so the eight input combinations cover 800 ns. The expected output for every step is listed in Table 1. The simulation waveform in Fig 5 shows A, B and Cin changing in binary counting order while Sum and Cout follow them.

Table 1: Simulation results of the 1-bit full adder

Time	A	B	Cin	Sum	Cout
0 – 100 ns	0	0	0	0	0
100 – 200 ns	0	0	1	1	0
200 – 300 ns	0	1	0	1	0
300 – 400 ns	0	1	1	0	1
400 – 500 ns	1	0	0	1	0
500 – 600 ns	1	0	1	0	1
600 – 700 ns	1	1	0	0	1
700 – 800 ns	1	1	1	1	1

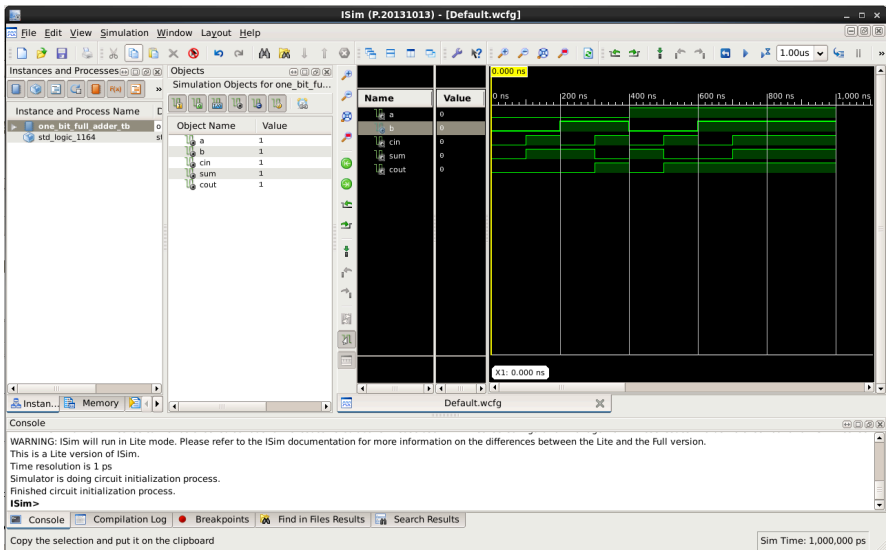


Fig 5: ISim waveform of the 1-bit full adder

Sum is 1 exactly when an odd number of inputs are 1 (the 2nd, 3rd, 5th and 8th steps). Cout is 1 exactly when at least two inputs are 1 (the 4th, 6th, 7th and 8th steps). The waveform in Fig 5 agrees with Table 1 in all eight cases, which confirms that the design behaves as a correct full adder.

7. Result

The 1-bit full adder was written in VHDL, synthesized and simulated. Its outputs matched the full-adder truth table for all eight input combinations, and the RTL schematic was generated successfully. The objective of the experiment was achieved.

8. GitHub Repository Commits

Repository commits link: https://github.com/Nazmul-SWE/VLSI

9. Conclusion

In this experiment I designed a 1-bit full adder in VHDL and verified it with a test bench that applies every input combination. I learned the structure of a VHDL design (entity and architecture) and how concurrent assignments describe combinational logic. Testing all eight combinations is enough to prove a full adder correct, because a three-input circuit has only eight possible input patterns. I also learned how to generate and read the RTL schematic in ISE and how to read the ISim waveform. This module is the building block for larger adders, such as a ripple-carry adder, in later experiments.